Yggdrasil ERP — Server Architecture

Mimir Labs Technical Publication

Document: ML-WP-002 **Version:** 1.1
Author: Christopher Gaither **Date:** April 2026
Classification: Public

0.1 Executive Summary

Yggdrasil ERP is powered by a high-performance C++17 server built on the Qt 6 framework. The server provides RESTful HTTP and real-time WebSocket APIs to desktop, web, and mobile clients, managing 10 integrated business modules across 166 database tables. This white paper details the server's architecture, covering its boot sequence, authentication and authorization subsystem, middleware pipeline, service layer, route architecture, real-time event streaming, and operational characteristics.

0.2 1. Technology Foundation

Component	Specification
Language	C++17 (ISO/IEC 14882:2017)
Framework	Qt 6.2+ (Core, Sql, HttpServer, WebSockets, Network)
Build System	CMake 3.16+ with `CMAKE_AUTOMOC ON`
Compiler Support	GCC 10+, Clang 13+, MSVC 2019+
Database Driver	Qt QPSQL (PostgreSQL wire protocol)
Configuration	INI-format `server.conf` with section-based key-value pairs

The choice of C++17 with Qt 6 provides several advantages for an ERP server workload:

- **Deterministic memory management** — No garbage collection pauses; memory usage is predictable under sustained load.
- **Native multithreading** — Qt's thread pool maps directly to OS threads, avoiding the overhead of green-thread schedulers.
- **Mature SQL driver** — Qt's QPSQL driver provides prepared statements, parameterized queries, and connection pooling without a separate ORM dependency.
- **Cross-platform binary** — The same source compiles for Linux (production), Windows (development), and macOS without platform-specific code.

0.3 2. Boot Sequence

The server's `main.cpp` orchestrates initialization in a carefully ordered sequence designed for fail-fast behavior — if any critical subsystem fails to initialize, the server exits before accepting connections.

0.3.1 2.1 Initialization Order

```
1. QCoreApplication + QGuiApplication (headless or GUI mode)
2. Configuration loading (server.conf)
3. Logger initialization (structured JSON, file rotation)
4. Database connection pool (PostgreSQL)
5. Schema verification + migration runner
6. Seed data loader
7. Cache manager (in-memory TTL cache)
8. Metrics collector
9. Authentication manager (JWT + PBKDF2 + TOTP)
10. Secrets encryption service (AES-256-GCM)
11. B2B Event Hub (WebSocket server on :8081)
12. Notification service
13. State machine wiring
14. Redpanda relay (Kafka-protocol message broker)
15. HTTP server binding (:8080)
16. Route registration (31 route modules)
17. Server status window (optional GUI)
```

0.3.2 2.2 Dual-Mode Operation

The server supports both headless (daemon) and GUI modes:

- **Headless mode** — `QCoreApplication` for production deployments. No display server required.

- **GUI mode** — `QGuiApplication` with a `ServerStatusWindow` displaying real-time connection counts, request rates, and log output. Used during development and operator-attended deployments.

The GUI mode includes an operator login dialog for secure local access, providing a dedicated interface for server management without exposing administrative APIs.

0.3.3 2.3 Fail-Fast Design

Critical initialization failures — database connection failure, configuration parse errors, port binding conflicts — cause immediate exit with structured error logging. The server never enters a partially-initialized state where some routes work but others fail silently.

0.4 3. Authentication and Authorization

0.4.1 3.1 Authentication Pipeline

Yggdrasil implements a multi-factor authentication system with progressive security hardening:

Layer	Implementation
Password hashing	PBKDF2-HMAC-SHA256, 600,000 iterations, 32-byte random salt
Password comparison	Constant-time comparison to prevent timing attacks
Password complexity	Minimum length, mixed case, digits, special characters required
Password history	Prevents reuse of the last N passwords per user
Multi-factor auth	TOTP (RFC 6238) with 30-second time step, SHA-1 HMAC, 6-digit codes
Recovery codes	10 single-use recovery codes generated on TOTP enrollment
Account lockout	Progressive: 15min → 30min → 1hr → 4hr → 24hr (5 tiers)
Session tokens	JWT (HS256) with 1-hour access tokens and 7-day refresh tokens

0.4.2 3.2 Token Architecture

Authentication produces two tokens:

- **Access token** — Short-lived JWT (1 hour) containing user ID, tenant ID, role, and module permissions. Transmitted via `HttpOnly; Secure; SameSite=Lax` cookie.
- **Refresh token** — Long-lived JWT (7 days) for silent token renewal. Stored in a separate cookie with stricter path scope.

The `HttpOnly` flag prevents JavaScript access to tokens, mitigating XSS-based token theft. The

`Secure` flag ensures tokens are only transmitted over HTTPS. `SameSite=Lax` prevents CSRF attacks on state-changing requests.

0.4.3 3.3 Role-Based Access Control (RBAC)

Every authenticated request passes through the `enforceRbac()` middleware before reaching route handlers:

1. **Module detection** — The middleware auto-detects the target module from the request path and database table name.
2. **Permission check** — The user's role is compared against the module's required permission level.
3. **Fail-closed** — Unrecognized routes are denied by default. This prevents newly added endpoints from accidentally being exposed without authorization.

Role hierarchy: - **DevAdmin** — Full system access, cross-tenant operations, infrastructure management - **TenantAdmin** — Full access within their tenant, user management, configuration - **Manager** — Module-level access with approval authority - **User** — Standard CRUD operations within assigned modules - **ReadOnly** — View-only access

0.4.4 3.4 Centralized Authentication

The `CentralAuthManager` handles multi-tenant authentication against a centralized VPS user registry, enabling single sign-on across tenant deployments. This manager validates credentials against the central registry, provisions local user records on first login, and synchronizes role changes.

0.5 4. Middleware Pipeline

Every HTTP request passes through a middleware pipeline before reaching the route handler:

0.5.1 4.1 Request Flow

```
Incoming Request
  → CORS Headers
  → Rate Limiter
  → JWT Validation
  → Tenant Resolution
  → RBAC Enforcement
  → Route Handler
  → Response Serialization
  → Audit Logging
```

0.5.2 4.2 Rate Limiting

The `RateLimiter` middleware provides per-endpoint API throttling:

- **Token bucket algorithm** — Each endpoint has a configurable bucket size and refill rate.
- **Per-IP tracking** — Rate limits are tracked per client IP address.
- **Response headers** — `X-RateLimit-Remaining` and `X-RateLimit-Reset` headers inform clients of their quota status.
- **429 responses** — Exceeded limits return `429 Too Many Requests` with a `Retry-After` header.

0.5.3 4.3 CORS Configuration

Cross-Origin Resource Sharing headers are applied to all responses:

- Configurable allowed origins via `server.conf`
- Preflight (`OPTIONS`) requests are handled automatically
- Credentials are permitted for cookie-based authentication

0.6 5. Route Architecture

The server employs a modular route architecture with 31 route modules spanning 10 business domains. Each route module is a self-contained compilation unit that registers its endpoints during server boot.

0.6.1 5.1 Route Modules

Domain	Route Files	Approximate Endpoints
CRM	CrmRoutes.cpp	20+
Sales	SalesRoutes.cpp	25+
Purchasing	PurchasingRoutes.cpp	30+
Manufacturing	ManufacturingRoutes.cpp	35+
Warehouse	WarehouseRoutes.cpp	25+
Finance	FinanceRoutes.cpp	30+
Projects	ProjectRoutes.cpp	20+
PLM	PlmRoutes.cpp	30+
Quality	QualityRoutes.cpp	25+
Service	ServiceRoutes.cpp	25+
HR	HrRoutes.cpp	15+
Admin	AdminRoutes.cpp	15+
Auth	AuthRoutes.cpp	10+

Domain	Route Files	Approximate Endpoints
Workflow	WorkflowRoutes.cpp	15+
Approval	ApprovalRoutes.cpp	10+
Form Builder	FormRoutes.cpp	18+
Integration	IntegrationRoutes.cpp	10+
Notifications	NotificationRoutes.cpp	10+
Assets	AssetRoutes.cpp	14+
Logistics	LogisticsRoutes.cpp	15+
Data Management	DataManagementRoutes.cpp	10+
Search	SearchRoutes.cpp	5+
Total	**31 files**	**~400+ endpoints**

0.6.2 5.2 Route Helper Utilities

`RouteHelpers` provides common patterns used across all route modules:

- `filteredListResponse()` — Standardized paginated list queries with `?search=`, `?page=`, `?limit=`, `?sort=`, `?order=` parameters. Tenant-scoped by default.
- `singleResponse()` — Single-entity retrieval with tenant isolation.
- `logChange()` — Audit trail entry for data mutations (INSERT, UPDATE, DELETE, STATUS_CHG).
- `resolveUserId()` / `resolveTenantId()` — Extract authenticated context from the request.

0.6.3 5.3 OpenAPI Documentation

The server auto-generates an OpenAPI 3.0.3 specification served at `/api/docs` with an embedded Swagger UI. The specification covers:

- 235 API paths
- 319 operations
- 301 component schemas
- Full request/response type definitions
- `additionalProperties: false` enforcement on all create/update schemas (Data DMZ pattern)

0.7 6. Service Layer

0.7.1 6.1 State Machine Engine

The `StateMachine` service enforces valid status transitions across 23 entity types:

- **Transition validation** — Each entity type has a defined state graph. Transitions not in the

graph are rejected with a 409 Conflict response.
- **Atomic execution** — Status updates, audit logging, and event emission occur within a single transaction.
- **Event emission** — Every valid transition emits a `state_transition` event to the B2B Event Hub for real-time notification.
- **Audit trail** — All transitions are logged to `audit_change_log` with `STATUS_CHG` action and JSONB before/after values.

Covered entity types include: sales quotes, sales orders, sales invoices, purchase orders, work orders, work order operations, MRP planned orders, PLM parts, engineering change requests, eBOMs, quality reports (8D, CAPA, NCR, audits), service tickets, RMAs, maintenance orders, journal entries, AP bills, finance invoices and bills, workflow instances, approval requests, form templates, form submissions, and integration dead letters.

0.7.2 6.2 Cache Manager

In-memory caching with TTL-based expiration:

- **Key pattern** — `module:entity:id` (e.g., `crm:account:550e8400-` …)
- **Maximum size** — 512 MB (configurable)
- **Eviction** — LRU (Least Recently Used) when capacity is reached
- **Invalidation** — Automatic on write operations to the cached entity

0.7.3 6.3 Metrics Collector

Operational metrics for monitoring and performance analysis:

- **Request counters** — Per-endpoint request counts and error rates
- **Duration tracking** — Response time histograms per route
- **Exposed at** — `GET /metrics` for integration with external monitoring tools

0.7.4 6.4 Notification Service

Server-push notifications via WebSocket:

- **Channels** — Per-tenant notification channels with user filtering
- **Types** — System alerts, workflow events, approval requests, warranty expirations
- **Preferences** — Per-user notification preferences (WebSocket, email, or both)
- **Email fallback** — SMTP client with STARTTLS/AUTH LOGIN for email delivery

0.7.5 6.5 Secrets Encryption

The `SecretsCrypto` service provides application-level encryption:

- **Algorithm** — AES-256-GCM (authenticated encryption)
- **Use cases** — Encrypting sensitive configuration values, API keys, and credentials at rest
- **Key management** — Encryption key stored separately from encrypted data

---

0.8 7. Database Layer

0.8.1 7.1 Repository Pattern

The `Repository` class (in `common/`) provides a clean abstraction over Qt's QSqlDatabase:

- **Connection pooling** — Thread-safe connection management with configurable pool size (default: 100 connections)
- **Prepared statements** — All queries use parameterized prepared statements to prevent SQL injection
- **Transaction support** — `ScopedTransaction` RAII wrapper for automatic commit/rollback
- **Multi-tenancy** — Tenant ID injection on all tenant-scoped queries

0.8.2 7.2 Migration System

Sequential numbered migrations (`001_*.sql` through `033_*.sql`) with:

- **Forward migrations** — Applied automatically on server boot
- **Rollback support** — `.down.sql` files for each migration
- **Checksum verification** — Detects modified migrations
- **Idempotency** — `IF NOT EXISTS` guards on DDL operations

0.8.3 7.3 Query Builder

The `QueryBuilder` provides safe dynamic query construction:

- `escape()` — Parameterizes values in WHERE clauses
- `escapeLike()` — Escapes LIKE pattern metacharacters
- **SQL injection prevention** — All user input passes through parameterized queries; no string concatenation in SQL construction

---

0.9 8. Real-Time Event Streaming

0.9.1 8.1 B2B Event Hub

The `B2BEventHub` (in `common/`) provides real-time bidirectional communication:

- **Protocol** — WebSocket (RFC 6455) on port 8081
- **Authentication** — JWT-based connection authentication
- **Tenant isolation** — Events are scoped to tenant channels; cross-tenant event leakage is impossible
- **Event queue** — In-memory queue with configurable depth (default: 1,000 events)
- **Retry logic** — Failed event deliveries retry up to 3 times with exponential backoff

0.9.2 8.2 Event Types

Event Type	Trigger
`state_transition`	Any status change via StateMachine
`notification`	System notifications, approval requests
`data_change`	CRUD operations on entity tables
`workflow_event`	Workflow step completions, instance transitions

0.9.3 8.3 Redpanda Relay

The `RedpandaRelay` service bridges the internal event system to an external Redpanda (Kafka-compatible) message broker:

- **Protocol** — Kafka wire protocol over TLS
- **Use cases** — External system integration, event archival, cross-deployment synchronization
- **Topics** — Tenant-scoped topic naming for isolation
- **Delivery guarantees** — At-least-once delivery with consumer offset tracking

0.10 9. Configuration

The server reads all runtime configuration from `server.conf`, organized by section:

Section	Key Settings
`[Database]`	Host, port, name, user, password, max connections (100)
`[Server]`	Listen address, HTTP port (8080), thread pool (10), CORS origins
`[B2B]`	WebSocket port (8081), event queue size (1000), retry attempts (3)
`[Security]`	JWT secret, session timeout (3600s), SSL cert/key paths
`[Logging]`	Level (INFO), file path, rotation (10 MB/file, 10 files)
`[Performance]`	Cache enabled/size (512 MB), query timeout (30s)
`[Email]`	SMTP host, port, TLS mode, auth credentials

Section	Key Settings
`[Redpanda]`	Broker address, TLS settings

0.11 10. Operational Characteristics

0.11.1 10.1 Health Monitoring

- `GET /health` — Returns server status with database connectivity probe (executes `SELECT 1`), response latency measurement, and component health indicators. Returns 503 when the database is unreachable.
- `GET /metrics` — Exposes request counters, error rates, and response time distributions.

0.11.2 10.2 Structured Logging

All log output uses structured JSON format with automatic file rotation:

- **Rotation** — 10 MB per file, 10 files retained (100 MB total)
- **Fields** — Timestamp, severity, component, message, and context-specific key-value pairs
- **Audit entries** — Security-relevant events (auth, authz, data mutations) are logged with user ID, action, and detail fields

0.11.3 10.3 Graceful Shutdown

The server handles SIGTERM/SIGINT with ordered teardown:

1. Stop accepting new connections
2. Drain active request queue
3. Close WebSocket connections with close frame
4. Flush pending events to Redpanda
5. Close database connections
6. Write final log entry

0.12 11. Build and Deployment

0.12.1 11.1 Build Pipeline

```
cd server && mkdir -p build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
```

CMake targets: - `YggdrasilServer` — Main server binary - `YggdrasilServerTests` — Catch2 unit test suite (enabled with `-DBUILD_TESTING=ON`)

0.12.2 11.2 CI/CD

GitHub Actions pipeline: - **server-build** — CMake + Ninja build in Docker container - **server-test** — Catch2 unit tests in Docker container - **schema-validate** — PostgreSQL schema + migrations + seed data verification

0.12.3 11.3 Deployment Topology

Production deployment on a Hetzner VPS (CPX31) with: - Docker Compose orchestration - Cloudflare Zero Trust tunnel for TLS termination and DDoS protection - No ports exposed directly to the internet (SSH on port 22 only) - `fail2ban` for SSH brute-force protection - Systemd timer for health checks every 5 minutes

---

0.13 12. Future Roadmap

- **TLS direct mode** — Enable server-side SSL for deployments without Cloudflare tunnels
- **Connection pooling v2** — pg_bouncer integration for horizontal scaling
- **gRPC interface** — Binary protocol option for high-throughput internal service communication
- **Plugin system** — Dynamic module loading for customer-specific extensions (under evaluation)
